

Experiment No:3

CODE:-

```
const express = require('express');
const bodyParser = require('body-parser');
const mongoose = require('mongoose');

const app = express();
app.use(bodyParser.json());
const PORT = 5000;

// MongoDB connection
mongoose.connect('mongodb://')
.then(() => console.log('Connected to MongoDB'))
.catch(err => console.error('Could not connect to MongoDB...', err));

// Schema
const userSchema = new mongoose.Schema({
  uname: { type: String, unique: true, required: true },
  password: { type: String, required: true }
});

const User = mongoose.model('Users', userSchema);

// Create user (Register)
app.post('/users', async (req, res) => {
  try {
    const { uname, password } = req.body;

    // Check if user already exists
    const existingUser = await User.findOne({ uname });
    if (existingUser) {
      return res.status(400).json({ error: 'Username already exists' });
    }

    // Hash password before saving

    const newUser = new User({ uname, password });
    const savedUser = await newUser.save();

    // Respond without password
    res.status(201).json({ _id: savedUser._id, uname: savedUser.uname });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
});  
// Login route  
app.post('/login', async (req, res) => {  
  try {  
    const { uname, password } = req.body;  
  
    // Find user by uname and match password directly  
    const user = await User.findOne({ uname, password });  
    if (!user) {  
      return res.status(400).json({ error: 'Invalid username or password' });  
    }  
  
    // Successful login  
    res.status(200).json({ message: 'Login successful', uname: user.uname });  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

OUTPUT:-

